Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет информационных технологий и управления

Кафедра информационных технологий автоматизированных систем

ОТЧЕТ
по лабораторной работе №1
по дисциплине
«Современные системы программирования»

Выполнила: ст. гр. 320604
К.А. Хаджинова

Проверила: Ю.О. Герман

Минск 2026

СОДЕРЖАНИЕ

1 Цель работы 3
2 Ход работы 4
2.1 Постановка задачи 4
2.2 Теоретические сведения 4
2.3 Подготовка среды разработки 5
2.4 Анализ *IL*-кода 5
2.5 Задание 1. Анализ *IL*-кода базовых конструкций 6
2.6 Генерация и выполнение *IL*-кода 6
2.7 Задание 2. Работа с *IL*-генератором 7
2.8 Задание 3. Создание анализатора кода 8
2.9 Исследование *JIT*-компиляции 8
2.10 Задание 4.1. Влияние *JIT*-компиляции на сложение элементов массива 10
3 Ответы на контрольные вопросы 11
Выводы 12
Приложение А (обязательное) Листинги программного кода 13

1 ЦЕЛЬ РАБОТЫ

Изучить структуру и принципы работы промежуточного языка *Intermediate Language* (*IL*) платформы *.NET*, получить навык анализа *IL*-кода методов на *C#*, освоить динамическую генерацию *IL*-кода средствами *ILGenerator* и *DynamicMethod*, а также исследовать влияние *JIT*-компиляции на производительность программ: сравнить обычную и ступенчатую (*tiered*) компиляцию, оценить задержку первого вызова метода и стоимость вызова через делегат и виртуальный метод.

2 ХОД РАБОТЫ

Работа выполнена в консольном приложении *C#* для платформы *.NET* 10 (*SDK* 10.0.401). Полный текст программы приведен в приложении А. Ниже последовательно описаны постановка задачи, теоретическая основа, реализация заданий и результаты проверки.

2.1 Постановка задачи

Для изучения *IL*-кода и *JIT*-компиляции в рамках работы требуется выполнить следующие задачи:

1 проанализировать *IL*-код базовых конструкций языка *C#* (арифметика, условие, цикл) и описать назначение команд;

2 создать динамические методы с помощью *ILGenerator*: сложение, факториал в виде цикла и рекурсии, максимум из трех чисел, оценка по значению;

3 реализовать анализатор кода, выводящий *IL*-байты метода и сравнивающий производительность динамических методов с обычными с помощью *BenchmarkDotNet*;

4 исследовать *JIT*-компиляцию: время первого и повторного вызовов, режимы *Tiered Compilation*, вызов через делегат и виртуальный метод;

5 оценить влияние *JIT*-компиляции на сложение элементов массива.

2.2 Теоретические сведения

IL (*Intermediate Language*) – промежуточный язык, на который компилируется весь код экосистемы *.NET* (*C#*, *F#* и другие). Язык имеет стековую архитектуру: операции берут операнды с вершины стека вычислений и возвращают результат в тот же стек. Объектная модель *IL* поддерживает классы, интерфейсы и наследование.

JIT-компиляция (*Just-In-Time*) – компиляция *IL*-кода в машинный код в процессе выполнения программы. Режим *Normal JIT* компилирует метод при первом вызове. Режим *Tiered Compilation* адаптивный: метод сначала быстро компилируется без глубоких оптимизаций, а часто вызываемые методы затем перекомпилируются с оптимизацией.

Класс *DynamicMethod* вместе с *ILGenerator* позволяет создавать методы во время выполнения без исходного кода на *C#*. Для измерения скорости и выделяемой памяти применяется библиотека *BenchmarkDotNet*.

2.3 Подготовка среды разработки

Работа выполнена в консольном приложении *C#* на платформе *.NET*. Проект *Lab1* собран утилитой *dotnet* (*SDK* 10.0.401) с целевой платформой *net10.0*, как указано в методических указаниях. Библиотека *BenchmarkDotNet* версии 0.14.0 подключена пакетом *NuGet*. Файл проекта приведен в приложении А.

Для запуска заданий 1, 2 и 4 используется команда *dotnet run*, для задания 3 – команда *dotnet run -c Release -- bench*.

2.4 Анализ *IL*-кода

Программа из методических указаний получает тело метода через *GetMethodBody*() и выводит размер *IL*, число локальных переменных, размер стека и байты *IL* в шестнадцатеричном виде. Анализируются методы *SimpleMethod*, *MethodWithLoop* и *MethodWithCondition*. Результат показан на рисунке 2.1.

```
> dotnet run
=== Лабораторная работа: Анализ IL-кода ===


--- Анализ: Метод SimpleMethod ---
Размер IL: 4 байт
Локальные переменные: 0
Максимальный размер стека: 8

IL-инструкции (HEX):
02 03 58 2A


--- Анализ: Метод MethodWithLoop ---
Размер IL: 32 байт
Локальные переменные: 4
Максимальный размер стека: 2

IL-инструкции (HEX):
00 16 0A 17 0B 2B 08 06 07 58 0A 07 17 58 0B 07
02 FE 02 16 FE 01 0C 08 2D ED 06 0D 2B 00 09 2A



--- Анализ: Метод MethodWithCondition ---
Размер IL: 43 байт
Локальные переменные: 3
Максимальный размер стека: 2

IL-инструкции (HEX):
00 02 16 FE 02 0A 06 2C 08 72 91 01 00 70 0B 2B
18 02 16 FE 04 0C 08 2C 08 72 A3 01 00 70 0B 2B
08 72 B5 01 00 70 0B 2B 00 07 2A
```

Рисунок 2.1 – Анализ *IL*-кода методов примера

Метод *SimpleMethod* занимает 4 байта: 02 03 58 2*A* соответствует командам *ldarg.0*, *ldarg.1*, *add* и *ret*. Метод с циклом занимает 32 байта и использует четыре локальные переменные, метод с условием – 43 байта и три переменные. Значение максимального стека 8 для коротких методов выдается из-за особенностей компактного заголовка тела метода.

2.5 Задание 1. Анализ *IL*-кода базовых конструкций

Для задания написан метод нахождения произведения двух целых чисел. Его *IL*-код получен тем же методом *AnalyzeIL*. Исходный код метода приведен в приложении А.

Результат работы программы и *IL*-код метода показаны на рисунке 2.2.

```
> dotnet run
=== Задание 1: произведение двух целых чисел ===
Multiply(6, 7) = 42

--- Анализ: Метод Multiply ---
Размер IL: 4 байт
Локальные переменные: 0
Максимальный размер стека: 8

IL-инструкции (HEX):
02 03 5A 2A
```

Рисунок 2.2 – *IL*-код метода *Multiply*

Метод *Multiply*(6, 7) возвращает 42. Последовательность 02 03 5*A* 2*A* читается так: *ldarg.0* помещает в стек первый аргумент, *ldarg.1* – второй, *mul* извлекает два значения и помещает их произведение, *ret* возвращает значение с вершины стека. Отличие от метода сложения только в коде операции: 5*A* вместо 58.

2.6 Генерация и выполнение *IL*-кода

Методические указания содержат класс *ILGeneratorDemo* с тремя динамическими методами: сложение двух чисел, факториал через цикл и определение знака числа. Методы создаются через *DynamicMethod*, команды записываются вызовами *il.Emit*, а результат преобразуется в делегат. Результат вызова методов показан на рисунке 2.3.

```
> dotnet run
=== Динамическая генерация IL ===

DynamicAdd(15, 27) = 42
DynamicFactorial(5) = 120
DynamicFactorial(7) = 5040
DynamicGrade(10) = Positive
DynamicGrade(-5) = Negative
DynamicGrade(0) = Zero
```

Рисунок 2.3 – Результат вызова динамически созданных методов

Динамические методы возвращают те же значения, что и обычный код: сумма 15 и 27 равна 42, факториалы 5 и 7 равны 120 и 5040, знак чисел 10, минус 5 и 0 определяется как *Positive*, *Negative* и *Zero*.

2.7 Задание 2. Работа с *IL*-генератором

В задании 2.1 динамический метод вычисляет *n*! рекурсивно. Метод сравнивает *n* с единицей, при *n* не больше 1 возвращает 1, иначе умножает *n* на результат рекурсивного вызова самого себя командой *call*. В задании 2.2 метод находит максимум из трех чисел: значение первого аргумента сохраняется в локальной переменной, затем по очереди сравнивается со вторым и третьим. Код генерации приведен в приложении А.

Результаты вызова сгенерированных методов показаны на рисунке 2.4.

```
> dotnet run
=== Задание 2.1: n! рекурсивно ===

RecursiveFactorial(5) = 120
RecursiveFactorial(7) = 5040

=== Задание 2.2: максимум из трёх чисел ===

DynamicMax(3, 9, 5) = 9
DynamicMax(10, 2, 7) = 10
DynamicMax(1, 4, 8) = 8
```

Рисунок 2.4 – Результаты заданий 2.1 и 2.2

Рекурсивный метод дает 120 для *n* равного 5 и 5040 для *n* равного 7, что совпадает с результатом итеративного метода. Метод максимума возвращает 9, 10 и 8 для наборов (3, 9, 5), (10, 2, 7) и (1, 4, 8).

2.8 Задание 3. Создание анализатора кода

Для пунктов 2.1 и 2.2 созданы классы-анализаторы *BenchmarkDotNet*. Анализатор факториала сравнивает рекурсивный метод на *C#*, динамический метод с циклом и динамический рекурсивный метод. Анализатор максимума сравнивает метод на *C#* и динамический метод. Входные числа хранятся в полях класса, чтобы компилятор не вычислил результат заранее. Код анализатора факториала приведен в приложении А.

Результаты запуска анализаторов в режиме *Release* показаны на рисунке 2.5.

```
> dotnet run -c Release -- bench
BenchmarkDotNet v0.14.0, Windows 11 (10.0.26200.9550)
.NET SDK 10.0.401
| Method           | Mean      | Error     | StdDev    | Ratio | RatioSD | Allocated | Alloc Ratio |
|----------------- |----------:|----------:|----------:|------:|--------:|----------:|------------:|
| CSharpRecursive  |  8.333 ns | 0.9259 ns | 0.0507 ns |  1.00 |    0.01 |         - |          NA |
| DynamicLoop      |  5.087 ns | 2.9562 ns | 0.1620 ns |  0.61 |    0.02 |         - |          NA |
| DynamicRecursive | 16.388 ns | 3.9041 ns | 0.2140 ns |  1.97 |    0.02 |         - |          NA |
BenchmarkDotNet v0.14.0, Windows 11 (10.0.26200.9550)
.NET SDK 10.0.401
| Method     | Mean      | Error     | StdDev    | Ratio | RatioSD | Allocated | Alloc Ratio |
|----------- |----------:|----------:|----------:|------:|--------:|----------:|------------:|
| CSharpMax  | 0.0893 ns | 0.1219 ns | 0.0067 ns |  1.00 |    0.09 |         - |          NA |
| DynamicMax | 1.9968 ns | 0.5818 ns | 0.0319 ns | 22.44 |    1.43 |         - |          NA |
```

Рисунок 2.5 – Результаты *BenchmarkDotNet* для заданий 3.1 и 3.2

Для факториала от 10 среднее время составило около 8,3 нс для рекурсивного метода на *C#*, 5,1 нс для динамического цикла и 16,4 нс для динамического рекурсивного метода. Цикл быстрее рекурсии, так как не тратит время на вызов метода на каждом шаге. Для максимума из трех чисел динамический метод выполняется примерно за 2,0 нс, метод на *C#* – менее чем за 0,1 нс, потому что *JIT*-компилятор встраивает его в вызывающий код, а динамический метод вызывается через делегат. Выделения памяти нет ни в одном варианте. Погрешности измерений (столбец *Error*) велики, так как использована короткая конфигурация запуска.

2.9 Исследование *JIT*-компиляции

Пример из методических указаний измеряет время вызова методов *NormalMethod*, *VirtualMethod* и *DelegateMethod* класса *JitBenchmark* через рефлексию после прогрева *JIT*, затем выводит сведения о методе *Calculator.Add* и профиль трех способов вызова на 10 миллионов итераций. Результат измерения и инспекции показан на рисунке 2.6.

```
> dotnet run
=== JIT Бенчмарк (упрощенная версия) ===

Тестирование: NormalMethod
  Время: 10 мс
  Тики: 103,801

Тестирование: VirtualMethod
  Время: 44 мс
  Тики: 441,437

Тестирование: DelegateMethod
  Время: 31 мс
  Тики: 314,822


=== Инспекция метода: Add ===
```

Рисунок 2.6 – Измерение *JIT*-бенчмарка и инспекция метода *Add*

Обычный метод выполняется быстрее виртуального метода и вызова через делегат (10, 44 и 31 мс за 10 000 вызовов). Метод *Add* объявлен виртуальным, его *IL* занимает 9 байт.

Продолжение вывода программы, в котором определяется режим компиляции, показано на рисунке 2.7.

```
Адрес метода: 0x00007FFCC6B99950
Размер IL: 9 байт
Локальные переменные: 1
Максимальный стек: 2

Атрибуты метода:
  - IsVirtual: True
  - IsStatic: False
  - IsAbstract: False
  - IsFinal: False

=== Сравнение режимов JIT ===

TieredCompilation: ВКЛЮЧЕН

Для изменения режимов добавьте в .csproj:
<TieredCompilation>false</TieredCompilation>
```

Рисунок 2.7 – Режим *JIT*-компиляции (продолжение вывода)

Режим *Tiered Compilation* включен по умолчанию, поэтому методы сначала компилируются быстро, а затем при частых вызовах оптимизируются повторно.

2.10 Задание 4.1. Влияние *JIT*-компиляции на сложение элементов массива

Метод *SumArray* суммирует 10000 целых чисел массива. Первый вызов включает *JIT*-компиляцию метода, последующие вызовы выполняют уже скомпилированный код. Для сравнения время первого вызова сопоставляется со средним временем 1000 повторных вызовов. Код задания приведен в приложении А.

Результаты показаны на рисунке 2.8.

```
> dotnet run
=== Профилирование производительности ===

Прямой вызов: 12 мс
Через делегат: 34 мс
Через виртуальный метод: 34 мс

=== Задание 4.1: сумма элементов массива (10000 элементов) ===

Сумма = 49995000
Первый вызов (с JIT-компиляцией): 186.5 мкс
Повторный вызов (без JIT-компиляции): 28.2 мкс
```

Рисунок 2.8 – Результаты профилирования и задания 4.1

Прямой цикл выполнился за 12 мс, вызов через делегат – за 34 мс, через виртуальный метод – за 34 мс. Сумма элементов массива равна 49995000. Первый вызов *SumArray* занял 186,5 мкс, повторные вызовы в среднем 28,2 мкс, то есть *JIT*-компиляция удлинила первый вызов примерно в 6,6 раза.

3 ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

1. Что такое *IL*-код и зачем он нужен?

IL-код – промежуточный байт-код платформы *.NET*, в который компилируются все языки платформы. Он не зависит от процессора и операционной системы, благодаря чему одна сборка выполняется на разных платформах после *JIT*-компиляции. В *IL* существуют инструкции загрузки и сохранения (*ldarg*, *ldloc*, *stloc*, *ldc*, *ldstr*), арифметики (*add*, *sub*, *mul*), переходов (*br*, *bgt*, *blt*), вызовов (*call*, *callvirt*, *ret*) и работы с объектами (*newobj*, *ldfld*, *stfld*). Команда *stloc* сохраняет значение с вершины стека в локальную переменную, а *starg* – в аргумент метода.

2. В чем разница между *Normal JIT* и *Tiered Compilation*?

При *Normal JIT* метод компилируется один раз при первом вызове с полной оптимизацией. При *Tiered Compilation* метод сначала компилируется быстро и без глубокой оптимизации, а затем часто вызываемые методы перекомпилируются в оптимизированный вариант. Это ускоряет запуск программы и сохраняет высокую скорость горячих участков.

3. Что такое холодный старт и как его уменьшить?

Холодный старт – это медленное выполнение первых вызовов из-за *JIT*-компиляции методов и загрузки сборок. Его уменьшают предварительной компиляцией *ReadyToRun* или *Native AOT*, быстрой начальной компиляцией (*Tiered Compilation*), прогревом методов перед ответственным участком.

4. Как оптимизируются часто вызываемые методы?

Среда выполнения считает вызовы методов. Метод, превысивший порог, перекомпилируется в более высокий уровень с оптимизациями: встраиванием коротких методов, оптимизацией циклов и использованием профиля выполнения.

5. Каковы преимущества генерации кода через *ILGenerator*?

Генерация позволяет создавать методы во время выполнения без исходного кода, подстраивать логику под данные и получать скорость, близкую к обычному коду, если делегат создан один раз и затем переиспользуется. Динамическая компиляция применяется в *ORM* и сериализаторах, *DI*-контейнерах, библиотеках прокси и мокирования, движках скриптов и вычислителях выражений.

ВЫВОДЫ

Изучены структура *IL* и основные команды: получены и разобраны байты *IL* методов сложения, умножения, цикла и условия. Создан набор динамических методов через *DynamicMethod* и *ILGenerator*, включая рекурсивный факториал и максимум из трех чисел. Анализаторы *BenchmarkDotNet* показали, что динамический метод максимума, вызываемый через делегат, уступает встраиваемому методу на *C#* на единицы наносекунд, а динамический цикл быстрее рекурсии. Исследование *JIT*-компиляции показало, что первый вызов метода дольше последующих примерно в 6,6 раза из-за затрат на компиляцию. Поставленная цель достигнута.

ПРИЛОЖЕНИЕ А
(обязательное)
Листинги программного кода

Листинг А.1 – Файл проекта *Lab1.csproj*

```
<Project Sdk="Microsoft.NET.Sdk">

  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net10.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
    <AllowUnsafeBlocks>true</AllowUnsafeBlocks>
  </PropertyGroup>

  <ItemGroup>
    <PackageReference Include="BenchmarkDotNet" Version="0.14.0" />
  </ItemGroup>

</Project>
```

Листинг А.2 – Метод *Multiply*

```
// Задание 1: произведение двух целых чисел
static int Multiply(int a, int b) => a * b;
```

Листинг А.3 – Рекурсивный факториал (задание 2.1)

```
// Задание 2.1: n! рекурсивно
// int Factorial(int n) { if (n <= 1) return 1; return n * Factorial(n - 1); }
public static Func<int, int> CreateRecursiveFactorialMethod()
{
    DynamicMethod dynamicMethod = new DynamicMethod(
        "RecursiveFactorial",
        typeof(int),
        new[] { typeof(int) },
        typeof(ILGeneratorDemo).Module
    );

    ILGenerator il = dynamicMethod.GetILGenerator();

    Label recurse = il.DefineLabel();

    // if (n > 1) goto recurse;
    il.Emit(OpCodes.Ldarg_0);
    il.Emit(OpCodes.Ldc_I4_1);
    il.Emit(OpCodes.Bgt, recurse);

    // return 1;
    il.Emit(OpCodes.Ldc_I4_1);
    il.Emit(OpCodes.Ret);
```

```
    // recurse: return n * Factorial(n - 1);
    il.MarkLabel(recurse);
    il.Emit(OpCodes.Ldarg_0);
    il.Emit(OpCodes.Ldarg_0);
    il.Emit(OpCodes.Ldc_I4_1);
    il.Emit(OpCodes.Sub);
    il.Emit(OpCodes.Call, dynamicMethod);   // рекурсивный вызов самого метода
    il.Emit(OpCodes.Mul);
    il.Emit(OpCodes.Ret);

    return (Func<int, int>)dynamicMethod.CreateDelegate(typeof(Func<int, int>));
}
```

Листинг А.4 – Максимум из трех чисел (задание 2.2)

```
// Задание 2.2: максимум из трёх чисел
// int Max(int a, int b, int c) { int max = a; if (b > max) max = b; if (c > max) max = c; return
max; }
public static Func<int, int, int, int> CreateMaxOfThreeMethod()
{
    DynamicMethod dynamicMethod = new DynamicMethod(
        "DynamicMax",
        typeof(int),
        new[] { typeof(int), typeof(int), typeof(int) },
        typeof(ILGeneratorDemo).Module
    );

    ILGenerator il = dynamicMethod.GetILGenerator();

    Label skipB = il.DefineLabel();
    Label skipC = il.DefineLabel();

    il.DeclareLocal(typeof(int));  // max

    // max = a;
    il.Emit(OpCodes.Ldarg_0);
    il.Emit(OpCodes.Stloc_0);

    // if (b <= max) goto skipB; max = b;
    il.Emit(OpCodes.Ldarg_1);
    il.Emit(OpCodes.Ldloc_0);
    il.Emit(OpCodes.Ble, skipB);
    il.Emit(OpCodes.Ldarg_1);
    il.Emit(OpCodes.Stloc_0);
    il.MarkLabel(skipB);

    // if (c <= max) goto skipC; max = c;
    il.Emit(OpCodes.Ldarg_2);
    il.Emit(OpCodes.Ldloc_0);
    il.Emit(OpCodes.Ble, skipC);
    il.Emit(OpCodes.Ldarg_2);
    il.Emit(OpCodes.Stloc_0);
    il.MarkLabel(skipC);
```

```
    // return max;
    il.Emit(OpCodes.Ldloc_0);
    il.Emit(OpCodes.Ret);

    return (Func<int, int, int>)dynamicMethod.CreateDelegate(typeof(Func<int, int, int>));
}
```

Листинг А.5 – Анализатор факториала

```
[MemoryDiagnoser]
[ShortRunJob]
public class FactorialAnalyzer
{
    private Func<int, int> _dynamicFactorial = null!;
    private Func<int, int> _dynamicRecursiveFactorial = null!;
    private int _n = 10;

    [GlobalSetup]
    public void Setup()
    {
        _dynamicFactorial = ILGeneratorDemo.CreateFactorialMethod();
        _dynamicRecursiveFactorial = ILGeneratorDemo.CreateRecursiveFactorialMethod();
    }

    [Benchmark(Baseline = true)]
    public int CSharpRecursive() => Factorial(_n);

    [Benchmark]
    public int DynamicLoop() => _dynamicFactorial(_n);

    [Benchmark]
    public int DynamicRecursive() => _dynamicRecursiveFactorial(_n);

    private static int Factorial(int n) => n <= 1 ? 1 : n * Factorial(n - 1);
}
```

Листинг А.6 – Задание 4.1

```
// Задание 4.1: сложение элементов массива из 10000 целых чисел
// Первый вызов включает JIT-компиляцию метода, последующие выполняют уже скомпилированный код
static void ArraySumTask()
{
    Console.WriteLine("\n=== Задание 4.1: сумма элементов массива (10000 элементов) ===\n");

    int[] array = new int[10000];
    for (int i = 0; i < array.Length; i++)
        array[i] = i;

    // Первый вызов: JIT-компиляция + выполнение
```

```
    long start = Stopwatch.GetTimestamp();
    long result = SumArray(array);
    long firstCall = Stopwatch.GetTimestamp() - start;
    Console.WriteLine($"Сумма = {result}");
    Console.WriteLine($"Первый вызов (с JIT-компиляцией): {firstCall * 1_000_000.0 /
Stopwatch.Frequency:F1} мкс");

    // Повторные вызовы: метод уже скомпилирован
    const int calls = 1000;
    start = Stopwatch.GetTimestamp();
    for (int i = 0; i < calls; i++)
        SumArray(array);
    long repeated = Stopwatch.GetTimestamp() - start;
    Console.WriteLine($"Повторный вызов (без JIT-компиляции): {repeated * 1_000_000.0 /
Stopwatch.Frequency / calls:F1} мкс");
}

[MethodImpl(MethodImplOptions.NoInlining)]
static long SumArray(int[] array)
{
    long sum = 0;
    for (int i = 0; i < array.Length; i++)
        sum += array[i];
    return sum;
}
```